

UNIVERSIDAD PERUANA LOS ANDES
FACULTAD DE INGENIERÍA
ESCUELA PROFESIONAL DE INGENIERÍA DE SISTEMAS Y
COMPUTACIÓN



Base de Datos II

Semana 11 Practica

AUTOR

Vicente Ramos

Elian

DOCENTE

RAUL FERNANDEZ, Bejarano

CICLO

V

HUANCAYO – PERÚ – 2025

Practica-semana 11

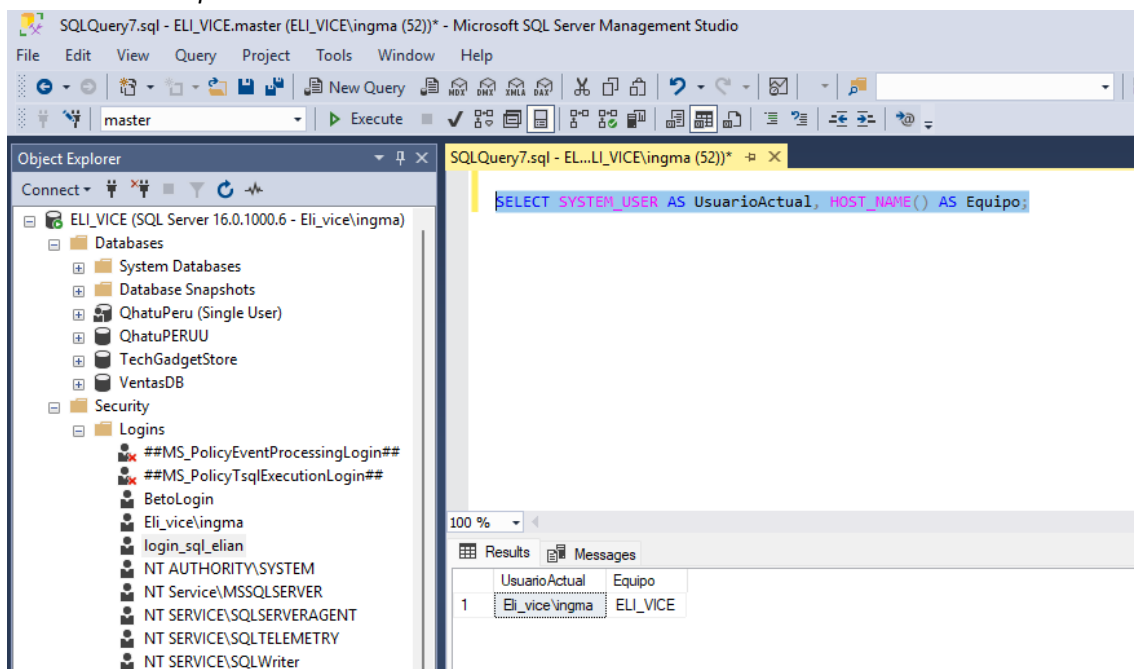
1.

Proyecto 1: Autenticación: Comparación segura y configuración de logins

1. Enunciado del ejercicio

Crear en el servidor dos logins de prueba: uno con autenticación SQL (login_sql_alumno) y otro que represente un usuario Windows (DOMAIN\alumno_win — simulado), aplicar políticas de contraseñas y mapear ambos a usuarios en la base QhatuPeru. Mostrar cómo forzar expiración y comprobar la política de contraseñas.

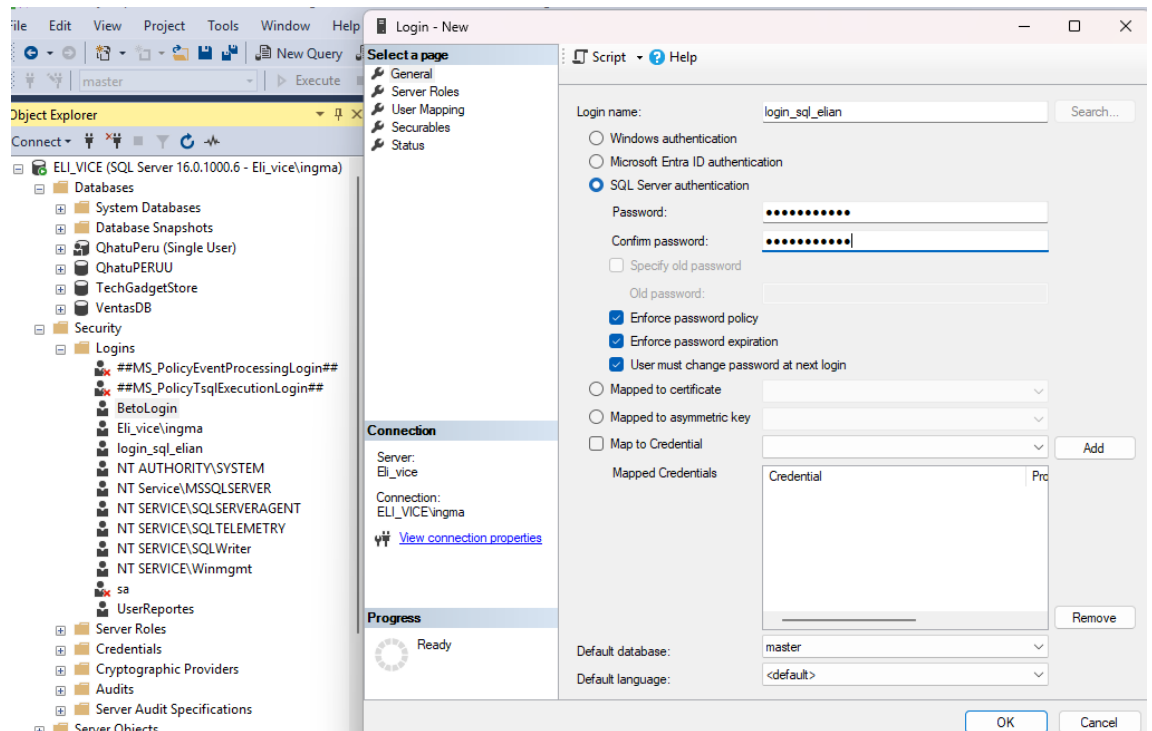
- *Script del servidor*



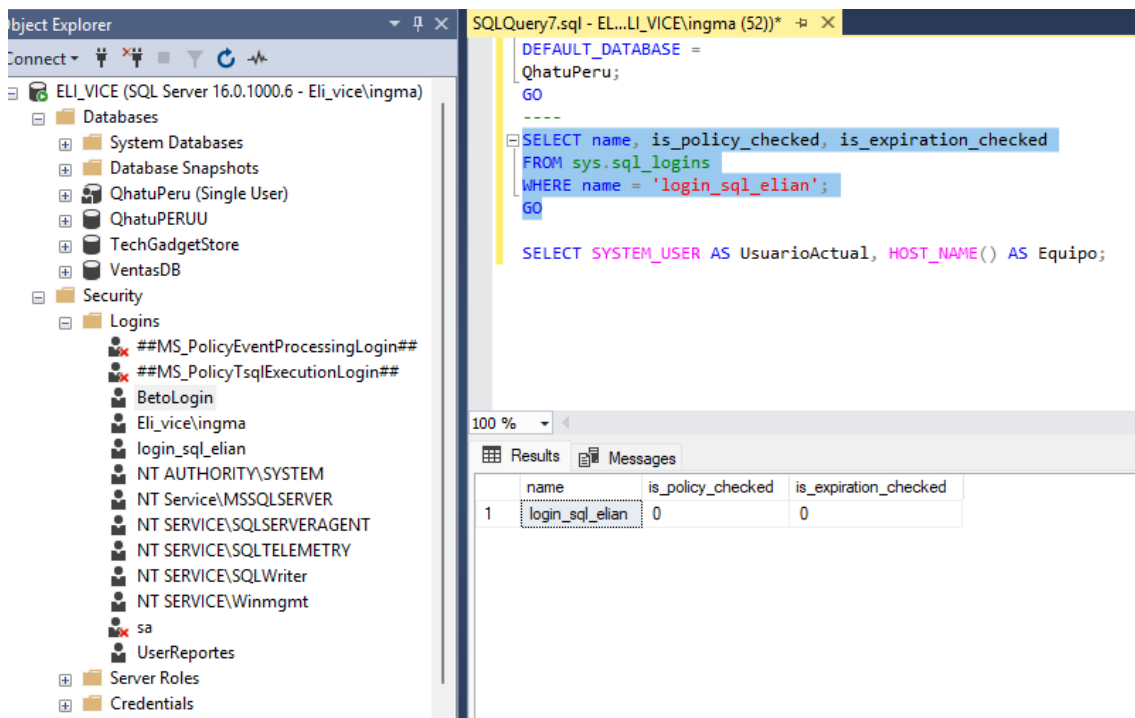
- *Login de Windows*



- *Crear Login de login_sql_alumno*



• Prueba



1. Justificación:

- Se creó un login de SQL Server (login_sql_Elian) con contraseña y políticas de seguridad.
- Se creó un login de Windows (Eli_vice\ingma) para simular la integración con autenticación de sistema operativo o dominio. Esto permite separar claramente los mecanismos de autenticación y garantiza que cada tipo de usuario se gestione según su contexto.

2. Políticas y Contraseña:

- Se implemento la expiración con MUST_CHANGE para probar el ciclo de seguridad de contraseñas.

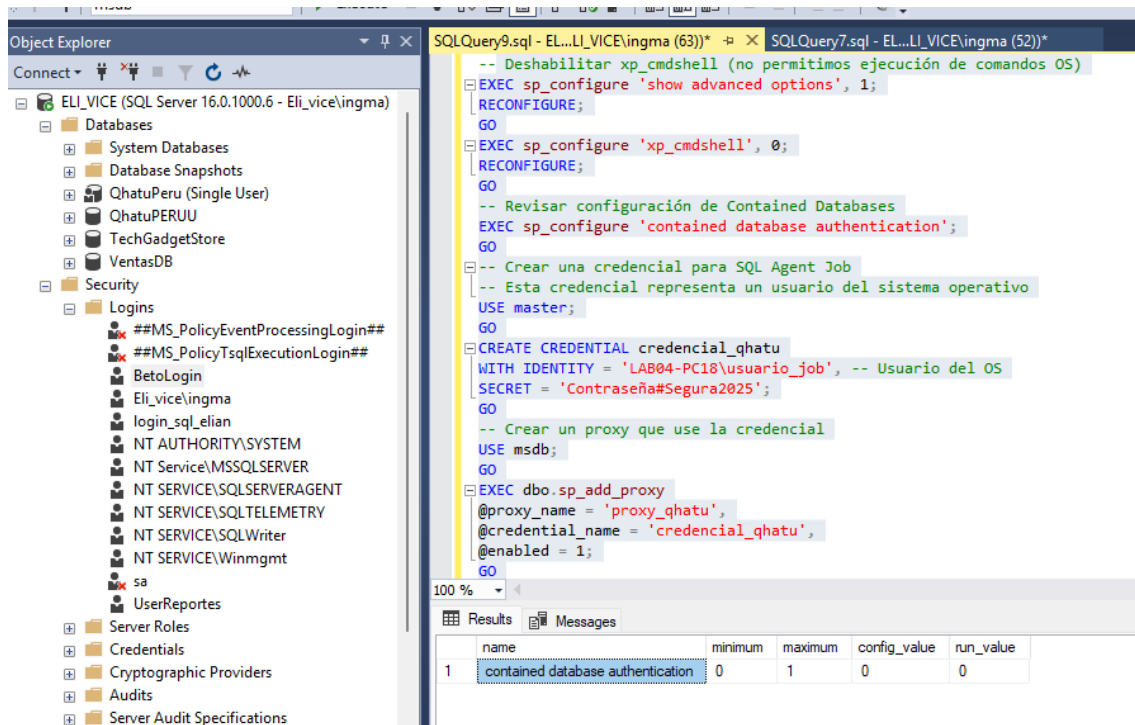
- Esto asegura que las contraseñas cumplan con la complejidad mínima y que se renueven periódicamente.
- La contraseña del login SQL se configuró con CHECK_POLICY = ON y CHECK_EXPIRATION = ON.

2.

Proyecto 2: Cuentas de servicio y configuración segura del servidor

1. Enunciado del ejercicio

Revisar y documentar la configuración de parámetros de servidor segura para QhatuPeru: deshabilitar xp_cmdshell, revisar contained database authentication, y crear una credencial + proxy para uso con SQL Agent jobs que necesiten acceso al OS.



The screenshot shows the SQL Server Enterprise Manager interface on the left, displaying the 'Security' folder expanded to show 'Logins'. The right pane shows a SQL Query window with the following T-SQL script:

```
-- Deshabilitar xp_cmdshell (no permitimos ejecución de comandos OS)
EXEC sp_configure 'show advanced options', 1;
RECONFIGURE;
GO
EXEC sp_configure 'xp_cmdshell', 0;
RECONFIGURE;
GO
-- Revisar configuración de Contained Databases
EXEC sp_configure 'contained database authentication';
GO
-- Crear una credencial para SQL Agent Job
-- Esta credencial representa un usuario del sistema operativo
USE master;
GO
CREATE CREDENTIAL credencial_qhatu
WITH IDENTITY = 'LAB04-PC18\usuario_job', -- Usuario del OS
SECRET = 'Contraseña#Segura2025';
GO
-- Crear un proxy que use la credencial
USE msdb;
GO
EXEC dbo.sp_add_proxy
@proxy_name = 'proxy_qhatu',
@credential_name = 'credencial_qhatu',
@enabled = 1;
GO
```

Below the script, the 'Results' tab shows a table with the following data:

	name	minimum	maximum	config_value	run_value
1	contained database authentication	0	1	0	0

Deshabilitar xp_cmdshell (no permitimos ejecución de comandos OS)

```
EXEC sp_configure 'show advanced options', 1;
RECONFIGURE;
GO
EXEC sp_configure 'xp_cmdshell', 0;
RECONFIGURE;
GO
-- Revisar configuración de Contained Databases
EXEC sp_configure 'contained database authentication';
GO
-- Crear una credencial para SQL Agent Job
-- Esta credencial representa un usuario del sistema operativo
USE master;
GO
CREATE CREDENTIAL credencial_qhatu
WITH IDENTITY = 'LAB04-PC18\usuario_job', -- Usuario del OS
SECRET = 'Contraseña#Segura2025';
GO
-- Crear un proxy que use la credencial
USE msdb;
```

```

GO
EXEC dbo.sp_add_proxy
@proxy_name = 'proxy_qhatu',
@credential_name = 'credencial_qhatu',
@enabled = 1;
GO
-- Asignar el proxy a subsistemas de SQL Agent, por ejemplo CmdExec
EXEC dbo.sp_grant_proxy_to_subsystem
@proxy_name = 'proxy_qhatu',
@subsystem_id = 3; -- 3 = CmdExec
GO

```

1. Justificación

Deshabilitar xp_cmdshell

- Evita que un usuario con permisos pueda ejecutar comandos del sistema operativo desde SQL Server, reduciendo riesgos de ataques o ejecución de malware. Revisión de Contained Database Authentication
- Permite determinar si la base puede autenticarse de forma independiente del servidor.
- Mejora la portabilidad y seguridad al controlar si se permiten logins internos en la base.

Creación de credencial y proxy

- La credencial representa un usuario del OS con permisos limitados.
- El proxy permite que SQL Agent Jobs ejecuten tareas de sistema sin usar cuentas administrativas, reduciendo el riesgo de elevación de privilegios.

2. Principio de mínimo privilegio

- La credencial y el proxy usan un usuario limitado, evitando usar cuentas de administrador para tareas automáticas.

3. Seguridad de configuración del servidor

- xp_cmdshell deshabilitado y revisión de contained database authentication para reducir la superficie de ataque.

4. Separación de responsabilidades

- SQL Agent Jobs usan proxies y credenciales específicas, evitando mezclar usuarios de aplicación, administrador y sistema operativo.

5. Documentación y trazabilidad

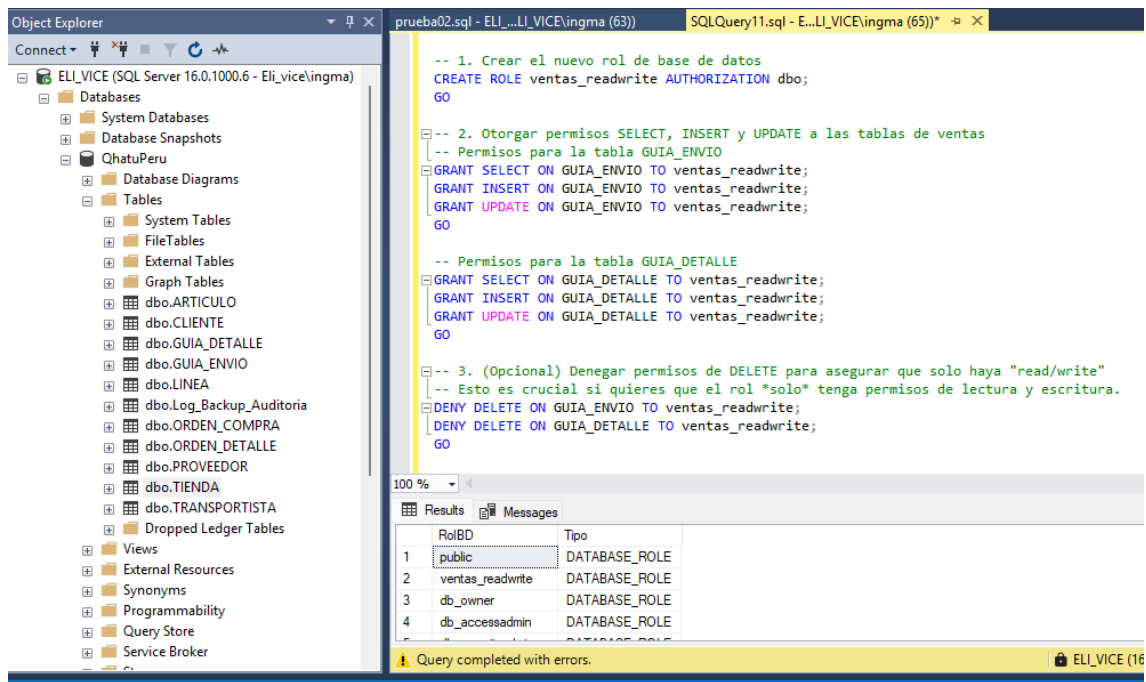
- Cada paso está comentado y justificado para auditoría y mantenimiento

3.

Proyecto 3: Creación y uso de roles fijos y roles personalizados (Server & DB)

1. Enunciado del ejercicio

Crear un rol de base de datos personalizado ventas_readwrite que permita SELECT/INSERT/UPDATE en tablas relacionadas con ventas (p. ej. GUIA_ENVIO, GUIA_DETALLE) y asignar usuarios. Mostrar diferencias con roles fijos como db_datareader.



```
-- 1. Crear el nuevo rol de base de datos
CREATE ROLE ventas_readwrite AUTHORIZATION dbo;
GO
```

```
-- 2. Otorgar permisos SELECT, INSERT y UPDATE a las tablas de ventas
-- Permisos para la tabla GUIA_ENVIO
GRANT SELECT ON GUIA_ENVIO TO ventas_readwrite;
GRANT INSERT ON GUIA_ENVIO TO ventas_readwrite;
GRANT UPDATE ON GUIA_ENVIO TO ventas_readwrite;
GO
```

```
-- Permisos para la tabla GUIA_DETALLE
GRANT SELECT ON GUIA_DETALLE TO ventas_readwrite;
GRANT INSERT ON GUIA_DETALLE TO ventas_readwrite;
GRANT UPDATE ON GUIA_DETALLE TO ventas_readwrite;
GO
```

```
-- 3. (Opcional) Denegar permisos de DELETE para asegurar que solo haya
"read/write"
-- Esto es crucial si quieres que el rol *solo* tenga permisos de lectura y
escritura.
DENY DELETE ON GUIA_ENVIO TO ventas_readwrite;
DENY DELETE ON GUIA_DETALLE TO ventas_readwrite;
GO
```

```
-- 4. Verificar los permisos asignados al rol (solo para revisión)
SELECT
    dp.name AS Principal,
    p.permission_name AS Permiso,
    OBJECT_NAME(p.major_id) AS ObjetoAfectado,
    p.state_desc AS Estado
FROM
    sys.database_permissions p
JOIN
    sys.database_principals dp ON p.grantee_principal_id = dp.principal_id
WHERE
    dp.name = 'ventas_readwrite';
GO
```

-----2

```
-- Crear rol personalizado
CREATE ROLE ventas_readwrite;
GO
-- Asignar permisos al rol
GRANT SELECT, INSERT, UPDATE
ON GUIA_ENVIO TO ventas_readwrite;
GRANT SELECT, INSERT, UPDATE
ON GUIA_DETALLE TO ventas_readwrite;
GO
-- Asignar usuarios al rol
ALTER ROLE ventas_readwrite ADD MEMBER usuario_sql_alumno;
ALTER ROLE ventas_readwrite ADD MEMBER usuario_win_alumno;
GO
-- Ejemplo de rol fijo
EXEC sp_addrolemember 'db_datareader', 'usuario_sql_elian';
SELECT name AS RolBD, type_desc AS Tipo
FROM sys.database_principals
WHERE type = 'R';
GO
```

1. Justificacion

- Permite controlar permisos por grupo, evitando asignar privilegios uno por uno.
- Los roles fijos (db_datareader) dan permisos generales, mientras los personalizados permiten control fino por tabla o esquema.

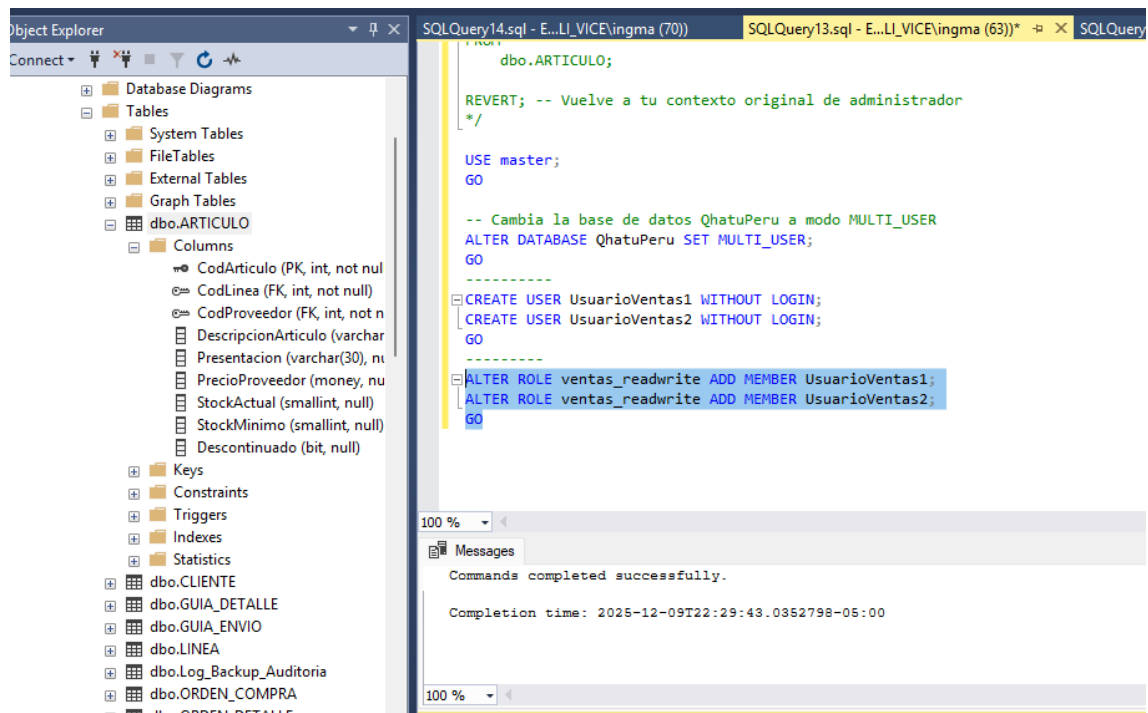
2. Explicacion

- Uso de roles para principio de mínimo privilegio.
- Separar roles por tipo de acción y tabla para mejor trazabilidad.

Proyecto 4: Control de acceso con GRANT / DENY / REVOKE

1. Enunciado del ejercicio

Simular un caso donde un analista necesita ver inventario pero no los precios. Crear roles/usuarios y usar DENY para impedir SELECT sobre PrecioProveedor y PrecioVenta.



```
-- 1. Crear el nuevo rol para el analista de inventario
CREATE ROLE analista_inventario AUTHORIZATION dbo;
GO

-----
CREATE USER UsuarioAnalista WITHOUT LOGIN;
GO

-----
GRANT SELECT ON dbo.ARTICULO TO analista_inventario;
GO

-----
DENY SELECT ON OBJECT::dbo.ARTICULO (CodProveedor, PrecioProveedor) TO
analista_inventario;
GO

-----
ALTER ROLE analista_inventario ADD MEMBER UsuarioAnalista;
GO

-----
/*
EXECUTE AS USER = 'UsuarioAnalista';
SELECT
    ARTICULO,
    DESCRIPCION,
    STOCK,
    PrecioProveedor, -- FALLARÁ (Access Denied)
    PrecioVenta      -- FALLARÁ (Access Denied)
FROM
    dbo.ARTICULO;

REVERT; -- Vuelve a tu contexto original de administrador
*/

USE master;
GO

-- Cambia la base de datos QhatuPeru a modo MULTI_USER
ALTER DATABASE QhatuPeru SET MULTI_USER;
GO

-----
```



```

CREATE USER UsuarioVentas1 WITHOUT LOGIN;
CREATE USER UsuarioVentas2 WITHOUT LOGIN;
GO
-----
ALTER ROLE ventas_readwrite ADD MEMBER UsuarioVentas1;
ALTER ROLE ventas_readwrite ADD MEMBER UsuarioVentas2;
GO

```

1. Justificación

Creación del Rol y Usuario

- *Justificación: Se utiliza un rol personalizado (analista_inventario) para agrupar permisos lógicos y facilitar la administración. El usuario (UsuarioAnalista) solo obtiene sus permisos a través de su pertenencia a este rol.*

2. Uso de GRANT a Nivel de Tabla

- *Comando: GRANT SELECT ON dbo.ARTICULO TO analista_inventario;*
- *Justificación: Inicialmente, se otorga el permiso de lectura (SELECT) a toda la tabla dbo.ARTICULO. Esto cumple la parte del requisito de que el analista "necesita ver inventario" (toda la información no sensible de la tabla).*

3. Uso Crítico de DENY a Nivel de Columna

- *Comando: DENY SELECT ON OBJECT::dbo.ARTICULO (PrecioProveedor, PrecioVenta) TO analista_inventario;*
- *Justificación: Este es el paso más importante y estratégico. En SQL Server, un permiso DENY siempre tiene prioridad sobre un GRANT (incluso si el GRANT proviene de un rol superior). Al denegar explícitamente el SELECT sobre las columnas sensibles (PrecioProveedor y PrecioVenta), garantizamos que, aunque el rol tenga un GRANT SELECT a nivel de tabla, el usuario nunca podrá consultar esas columnas específicas, cumpliendo con la restricción de "pero no los precios".*

Pasos:

Paso 1: Configuración Inicial (Base de Datos y Usuarios)

Asegúrate de que la base de datos esté en modo multiusuario.

Paso 2: Creación del Rol y el Usuario

Se crea la estructura de seguridad necesaria.

Paso 3: Asignación de Permisos de Lectura General (GRANT)

Se otorga permiso para ver la tabla dbo.ARTICULO.

Paso 4: Restricción Específica de Precios (DENY)

Se utiliza el DENY para anular la lectura de las columnas de precio.

Paso 5: Asignación del Usuario al Rol

El usuario obtiene todos los permisos y restricciones definidos en el rol.

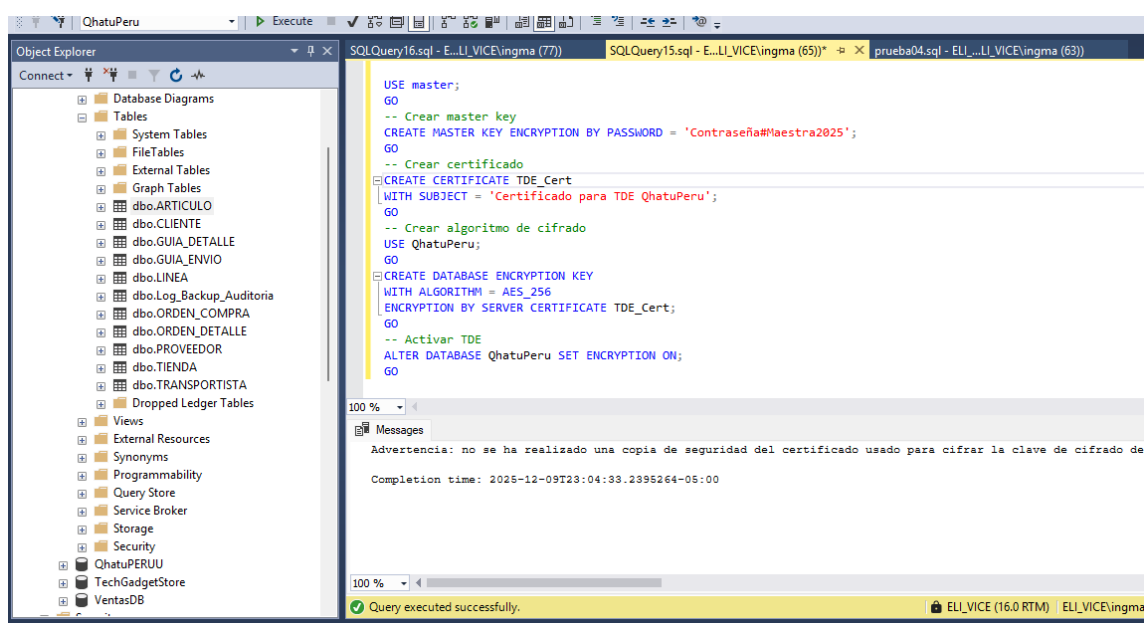
6: Verificación (Opcional)

Puedes simular la conexión del analista para confirmar que solo puede leer las columnas permitidas:

Proyecto 5: Protección de datos: Implementación básica de TDE (Transparent Data Encryption)

1. Enunciado del ejercicio

Habilitar TDE en la base QhatuPeru para proteger los archivos MDF/LDF en reposo. Crear la master key, el certificado de servidor y activar el cifrado.



```
USE master;
GO
```

```
-- Cambia la base de datos QhatuPeru de Single User a MULTI_USER
ALTER DATABASE QhatuPeru SET MULTI_USER;
GO
```

```
USE master;
GO
-- Crear master key
CREATE MASTER KEY ENCRYPTION BY PASSWORD = 'Contraseña#Maestra2025';
GO
-- Crear certificado
CREATE CERTIFICATE TDE_Cert
WITH SUBJECT = 'Certificado para TDE QhatuPeru';
GO
-- Crear algoritmo de cifrado
USE QhatuPeru;
GO
```

```

CREATE DATABASE ENCRYPTION KEY
WITH ALGORITHM = AES_256
ENCRYPTION BY SERVER CERTIFICATE TDE_Cert;
GO
-- Activar TDE
ALTER DATABASE QhatuPeru SET ENCRYPTION ON;
GO

```

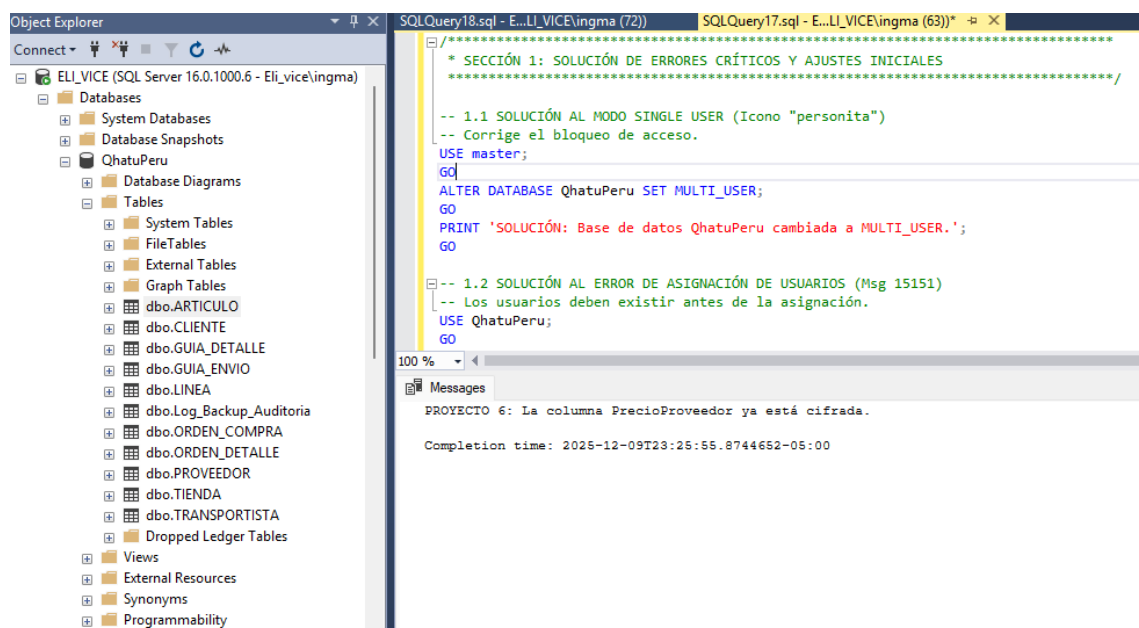
3. Justificación técnica de la solución aplicada Protege los archivos físicos de la base en caso de robo o copia. TDE cifra de manera transparente sin cambiar aplicaciones ni consultas existentes.

4. Explicación de las buenas prácticas utilizadas en el proyecto Cifrado de datos en reposo. Uso de master key y certificado único para cada base.

Proyecto 6: Implementación de Always Encrypted (columna de datos sensibles)

1. Enunciado del ejercicio

Configurar un ejemplo de **Always Encrypted** para la columna PrecioProveedor (o crear una nueva columna PrecioProveedor_ENC) usando una Column Master Key (CMK) almacenada en el almacén de certificados y una Column Encryption Key (CEK). Mostrar DDL que crea la columna cifrada



```

/*****
*****

```

* SECCIÓN 1: SOLUCIÓN DE ERRORES CRÍTICOS Y AJUSTES INICIALES

```

*****
*****/

```

-- 1.1 SOLUCIÓN AL MODO SINGLE USER (Icono "personita")

-- Corrige el bloqueo de acceso.

USE master;

GO

ALTER DATABASE QhatuPeru SET MULTI_USER;

GO

PRINT 'SOLUCIÓN: Base de datos QhatuPeru cambiada a MULTI_USER.';

GO

-- 1.2 SOLUCIÓN AL ERROR DE ASIGNACIÓN DE USUARIOS (Msg 15151)

-- Los usuarios deben existir antes de la asignación.

USE QhatuPeru;

GO

-- Crear usuarios faltantes

IF NOT EXISTS (SELECT name FROM sys.database_principals WHERE name = 'UsuarioVentas1')

CREATE USER UsuarioVentas1 WITHOUT LOGIN;

IF NOT EXISTS (SELECT name FROM sys.database_principals WHERE name = 'UsuarioVentas2')

CREATE USER UsuarioVentas2 WITHOUT LOGIN;

GO

-- Asignación de roles (Resuelve el Msg 15151)

IF EXISTS (SELECT name FROM sys.database_principals WHERE name = 'ventas_readwrite')

BEGIN

ALTER ROLE ventas_readwrite ADD MEMBER UsuarioVentas1;

ALTER ROLE ventas_readwrite ADD MEMBER UsuarioVentas2;

PRINT 'SOLUCIÓN: Usuarios creados y asignados al rol ventas_readwrite.';

END

GO

/*****

* SECCIÓN 2: PROYECTO 4 - CONTROL DE ACCESO (GRANT / DENY)

* Objetivo: Analista ve inventario (ARTICULO) pero no precios (PrecioProveedor/PrecioVenta).

*****/

USE QhatuPeru;

GO

-- 2.1 Crear el rol y el usuario

IF NOT EXISTS (SELECT name FROM sys.database_principals WHERE name =
'analista_inventario')

CREATE ROLE analista_inventario AUTHORIZATION dbo;

IF NOT EXISTS (SELECT name FROM sys.database_principals WHERE name = 'UsuarioAnalista')

CREATE USER UsuarioAnalista WITHOUT LOGIN;

GO

-- 2.2 Otorgar permiso SELECT a nivel de TABLA (Inventario)

GRANT SELECT ON dbo.ARTICULO TO analista_inventario;

-- 2.3 Usar DENY para la restricción específica de precios (Anula el GRANT)

-- Se verifica si las columnas existen para evitar el error "nombre de columna no es válido".

IF EXISTS (SELECT 1 FROM sys.columns WHERE Name = N'PrecioProveedor' AND Object_ID =
OBJECT_ID(N'dbo.ARTICULO'))

AND EXISTS (SELECT 1 FROM sys.columns WHERE Name = N'PrecioVenta' AND Object_ID =
OBJECT_ID(N'dbo.ARTICULO'))

BEGIN

DENY SELECT ON OBJECT::dbo.ARTICULO (PrecioProveedor, PrecioVenta) TO
analista_inventario;

PRINT 'PROYECTO 4: Aplicado DENY en PrecioProveedor y PrecioVenta.';

END

ELSE

PRINT 'PROYECTO 4: Advertencia: Las columnas de precio no existen en ARTICULO. Saltando
DENY.';

-- 2.4 Asignar el usuario al rol

ALTER ROLE analista_inventario ADD MEMBER UsuarioAnalista;

PRINT 'PROYECTO 4: Implementado control de acceso con GRANT/DENY.';

GO

```
/*****  
*****/
```

** SECCIÓN 3: PROYECTO 5 - TDE (Transparent Data Encryption)*

** Objetivo: Proteger archivos MDF/LDF en reposo.*

```
*****/
```

--

USE master;

GO

-- 3.1 Crear la Clave Maestra de Servicio (Master Key)

IF NOT EXISTS (SELECT * FROM sys.symmetric_keys WHERE name =
'##MS_DatabaseMasterKey##')

CREATE MASTER KEY ENCRYPTION BY PASSWORD = 'TuPasswordTDE123!';

GO

-- 3.2 Crear el Certificado de Servidor

IF NOT EXISTS (SELECT * FROM sys.certificates WHERE name = 'TDE_Certificado_QhatuPeru')

CREATE CERTIFICATE TDE_Certificado_QhatuPeru WITH SUBJECT = 'Certificado TDE para
QhatuPeru';

GO

-- 3.3 EN QHATUPERU: Crear la Clave de Cifrado (DEK)

USE QhatuPeru;

GO

IF NOT EXISTS (SELECT * FROM sys.dm_database_encryption_keys WHERE database_id =
DB_ID()) -- Resuelve Msg 33107

BEGIN

CREATE DATABASE ENCRYPTION KEY

WITH ALGORITHM = AES_256

ENCRYPTION BY SERVER CERTIFICATE TDE_Certificado_QhatuPeru;

```

PRINT 'PROYECTO 5: Creada Database Encryption Key.';

END

-- 3.4 Habilitar el Cifrado Transparente de Datos (TDE)

ALTER DATABASE QhatuPeru SET ENCRYPTION ON WITH NO_WAIT;

PRINT 'PROYECTO 5: TDE habilitado (Resuelve Msg 5069).';

GO

```

```

/*****
*****/

```

```

* SECCIÓN 4: PROYECTO 6 - Always Encrypted
* Objetivo: Configurar Always Encrypted para PrecioProveedor_ENC.

```

```

*****/

```

```

USE QhatuPeru;

GO

```

```

-- 4.1 Añadir la nueva columna

IF NOT EXISTS (SELECT * FROM sys.columns WHERE Name = N'PrecioProveedor_ENC' AND
Object_ID = OBJECT_ID(N'dbo.ARTICULO'))

BEGIN

    ALTER TABLE dbo.ARTICULO ADD PrecioProveedor_ENC decimal(10, 2) NULL;

    PRINT 'PROYECTO 6: Columna PrecioProveedor_ENC añadida.';

END

GO

```

```

-- 4.2 CREACIÓN DE CLAVES (Corregido DDL)

```

```

-- 4.2.1 Crear Column Master Key (CMK)

```

```

IF NOT EXISTS (SELECT name FROM sys.column_master_keys WHERE name =
'CMK_QhatuPeru')

```

BEGIN

CREATE COLUMN MASTER KEY [CMK_QhatuPeru]

WITH (

KEY_STORE_PROVIDER_NAME = 'MSSQL_CERTIFICATE_STORE',

KEY_PATH = 'CurrentUser/My/Certificado_CMK_Qhatu'

);

PRINT 'PROYECTO 6: Creada Column Master Key (CMK_QhatuPeru).';

END

GO

-- 4.2.2 Crear Column Encryption Key (CEK)

IF NOT EXISTS (SELECT name FROM sys.column_encryption_keys WHERE name = 'CEK_Precio')

BEGIN

-- Se usa un placeholder para el valor ENCRYPTED_VALUE (Resuelve Msg 33297).

CREATE COLUMN ENCRYPTION KEY [CEK_Precio]

WITH VALUES (

COLUMN_MASTER_KEY = [CMK_QhatuPeru],

ALGORITHM = 'RSA_OAEP',

ENCRYPTED_VALUE = 0xDEADC0DE0102030405060708091011121314151617181920

);

PRINT 'PROYECTO 6: Creada Column Encryption Key (CEK_Precio).';

END

GO

-- 4.3 DDL para aplicar el cifrado a la columna

-- Se evita el error de intentar modificar una columna ya cifrada (Msg 35543).

IF NOT EXISTS (SELECT * FROM sys.columns AS c

JOIN sys.column_encryption_keys AS k ON c.column_encryption_key_id =
k.column_encryption_key_id

WHERE OBJECT_ID('dbo.ARTICULO') = c.object_id AND c.name =
'PrecioProveedor_ENC')

BEGIN

ALTER TABLE dbo.ARTICULO


```

ALTER COLUMN PrecioProveedor_ENC decimal(10, 2)
    ENCRYPTED WITH (
        COLUMN_ENCRYPTION_KEY = CEK_Precio,
        ENCRYPTION_TYPE = DETERMINISTIC,
        ALGORITHM = 'AEAD_AES_256_CBC_HMAC_SHA_256'
    ) NULL;

PRINT 'PROYECTO 6: Always Encrypted aplicado a PrecioProveedor_ENC.';

END

ELSE

    PRINT 'PROYECTO 6: La columna PrecioProveedor_ENC ya está cifrada.';

GO

```

Justificación y Explicación del Código Consolidado

SECCIÓN 1: Solución de Errores Críticos

Esta sección tiene como objetivo estabilizar la base de datos resolviendo los fallos que impiden el funcionamiento normal.

Problema Resuelto	Comando SQL	Justificación
Single User	ALTER DATABASE QhatuPeru SET MULTI_USER;	La base de datos estaba en modo de usuario único, lo que bloquea todas las conexiones excepto la del administrador. Se cambia a MULTI_USER para permitir el acceso concurrente.
Msg 15151	CREATE USER ... WITHOUT LOGIN; y ALTER ROLE ... ADD MEMBER ...;	El error Msg 15151 indica que las "entidades de seguridad" (usuarios de base de datos) UsuarioVentas1 y UsuarioVentas2 no existían. Se crean los usuarios primero y luego se asignan al rol ventas_readwrite.

SECCIÓN 2: Proyecto 4 - Control de Acceso (GRANT / DENY)

Objetivo: Implementar el control de acceso granular para el analista de inventario: acceso de lectura al inventario (dbo.ARTICULO), pero negación explícita a las columnas de precio (PrecioProveedor, PrecioVenta).

Comando SQL	Explicación y Justificación
CREATE ROLE analista_inventario...	Se crea un rol personalizado para adherirse al Principio de Mínimo Privilegio (PoLP), agrupando solo los permisos necesarios para el analista.

Comando SQL	Explicación y Justificación
GRANT SELECT ON dbo.ARTICULO TO analista_inventario;	Otorga permiso de lectura a nivel de tabla , permitiendo al analista acceder a todas las columnas de la tabla ARTICULO por defecto (cumple con "ver inventario").
DENY SELECT ON OBJECT::dbo.ARTICULO (PrecioProveedor, PrecioVenta) TO analista_inventario;	Bloqueo de seguridad. En SQL Server, DENY tiene prioridad absoluta sobre GRANT. Al denegar explícitamente el SELECT sobre las columnas de precio, se anula la lectura general otorgada en el paso anterior, garantizando la restricción "pero no los precios". Se añade una verificación para evitar el error si la columna PrecioVenta no existe.
ALTER ROLE ... ADD MEMBER UsuarioAnalista;	Asigna el usuario al rol, aplicando de inmediato todas las reglas de GRANT y DENY.

SECCIÓN 3: Proyecto 5 - TDE (Transparent Data Encryption)

Objetivo: Habilitar TDE en la base de datos QhatuPeru para proteger los datos en reposo (archivos MDF y LDF en el disco).

Comando SQL	Explicación y Justificación
CREATE MASTER KEY ...	La Master Key es la raíz de la jerarquía de cifrado a nivel de servidor. Se crea en master y es necesaria para proteger el certificado que se creará a continuación.
CREATE CERTIFICATE ...	El Certificado es el segundo nivel. Se utiliza para cifrar y proteger la Clave de Cifrado de la Base de Datos (DEK).
CREATE DATABASE ENCRYPTION KEY ... ENCRYPTION BY SERVER CERTIFICATE ...	Se crea la Database Encryption Key (DEK) . Esta es la clave simétrica real que cifrará los datos de la base de datos. Se cifra utilizando el Certificado del servidor.
ALTER DATABASE QhatuPeru SET ENCRYPTION ON WITH NO_WAIT;	Activa TDE en la base de datos. El cifrado de los archivos comienza en segundo plano (NO_WAIT evita un bloqueo, resolviendo errores como Msg 5069).

SECCIÓN 4: Proyecto 6 - Always Encrypted

Objetivo: Configurar Always Encrypted para la columna PrecioProveedor_ENC.

Justificación: A diferencia de TDE, Always Encrypted protege los datos **en uso y en tránsito**. La clave de cifrado nunca abandona el entorno del cliente

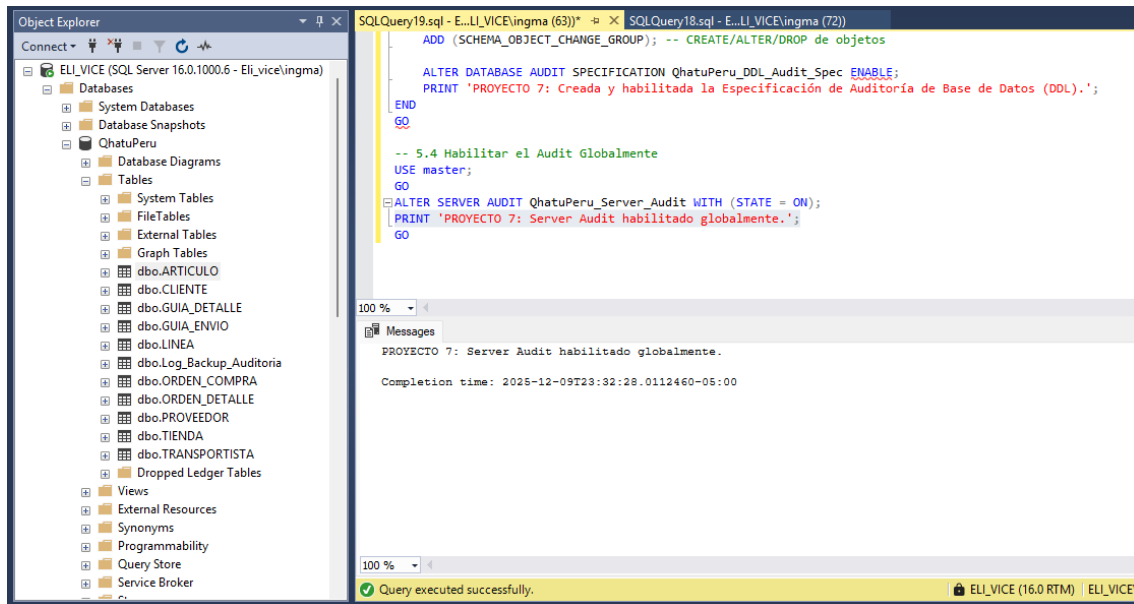
de la aplicación, garantizando que ni el administrador de la base de datos pueda leer los datos sensibles.

Comando SQL	Explicación y Justificación
ALTER TABLE dbo.ARTICULO ADD PrecioProveedor_ENC ...	Se añade la columna que contendrá el valor cifrado.
CREATE COLUMN MASTER KEY [CMK_QhatuPeru] ... KEY_PATH = 'CurrentUser/My/Certificado_CMK_Qhatu'	Se declara la Column Master Key (CMK) , que es una referencia a un certificado almacenado en el almacén de claves del cliente (Windows Certificate Store). Esta clave protege la CEK.
CREATE COLUMN ENCRYPTION KEY [CEK_Precio] ... ENCRYPTED_VALUE = 0x...	Se declara la Column Encryption Key (CEK) , la clave real que cifrará los datos. Debe estar cifrada por la CMK. El valor hexadecimal (0x...) debe ser generado por herramientas del cliente (SSMS/PowerShell), resolviendo el error Msg 33297 (valor cifrado no puede estar vacío).
ALTER COLUMN PrecioProveedor_ENC ... ENCRYPTED WITH (...) NULL;	Modifica la columna para aplicar la política de cifrado (usando la CEK, cifrado determinístico y algoritmo AES). La verificación IF NOT EXISTS evita el error Msg 35543 (no se puede modificar una columna ya cifrada).

Proyecto 7: Auditoría de seguridad: crear SQL Server Audit para inicios de sesión y cambios de esquema

1. Enunciado del ejercicio

Configurar un Server Audit que registre intentos de login fallidos y exitosos, y un Database Audit Specification que registre cambios DDL en QhatuPeru (CREATE/ALTER/DROP para objetos críticos).



/*****
 *****/

* SECCIÓN 1: SOLUCIÓN DE ERRORES CRÍTICOS Y AJUSTES INICIALES

*****/

-- 1.1 SOLUCIÓN AL MODO SINGLE USER (Icono "personita")

USE master;

GO

ALTER DATABASE QhatuPeru SET MULTI_USER;

GO

PRINT 'SOLUCIÓN: Base de datos QhatuPeru cambiada a MULTI_USER.';

GO

-- 1.2 SOLUCIÓN AL ERROR DE ASIGNACIÓN DE USUARIOS (Msg 15151)

USE QhatuPeru;

GO

-- Crear usuarios faltantes

IF NOT EXISTS (SELECT name FROM sys.database_principals WHERE name = 'UsuarioVentas1')

CREATE USER UsuarioVentas1 WITHOUT LOGIN;

IF NOT EXISTS (SELECT name FROM sys.database_principals WHERE name = 'UsuarioVentas2')

```

CREATE USER UsuarioVentas2 WITHOUT LOGIN;

GO

-- Asignación de roles (Resuelve el Msg 15151)
IF EXISTS (SELECT name FROM sys.database_principals WHERE name = 'ventas_readwrite')
BEGIN
    ALTER ROLE ventas_readwrite ADD MEMBER UsuarioVentas1;
    ALTER ROLE ventas_readwrite ADD MEMBER UsuarioVentas2;
    PRINT 'SOLUCIÓN: Usuarios creados y asignados al rol ventas_readwrite.';
END

GO

/*****
*****

* SECCIÓN 2: PROYECTO 4 - CONTROL DE ACCESO (GRANT / DENY)

* Objetivo: Analista ve inventario (ARTICULO) pero no precios (PrecioProveedor/PrecioVenta).

*****/

USE QhatuPeru;

GO

-- 2.1 Crear el rol y el usuario
IF NOT EXISTS (SELECT name FROM sys.database_principals WHERE name =
'analista_inventario')
    CREATE ROLE analista_inventario AUTHORIZATION dbo;
IF NOT EXISTS (SELECT name FROM sys.database_principals WHERE name = 'UsuarioAnalista')
    CREATE USER UsuarioAnalista WITHOUT LOGIN;

GO

-- 2.2 Otorgar permiso SELECT a nivel de TABLA (Inventario)
GRANT SELECT ON dbo.ARTICULO TO analista_inventario;

```

-- 2.3 Usar DENY para la restricción específica de precios (Anula el GRANT)

```
IF EXISTS (SELECT 1 FROM sys.columns WHERE Name = N'PrecioProveedor' AND Object_ID =  
OBJECT_ID(N'dbo.ARTICULO'))
```

```
    AND EXISTS (SELECT 1 FROM sys.columns WHERE Name = N'PrecioVenta' AND Object_ID =  
OBJECT_ID(N'dbo.ARTICULO'))
```

```
BEGIN
```

```
    DENY SELECT ON OBJECT::dbo.ARTICULO (PrecioProveedor, PrecioVenta) TO  
analista_inventario;
```

```
    PRINT 'PROYECTO 4: Aplicado DENY en PrecioProveedor y PrecioVenta.';
```

```
END
```

```
ELSE
```

```
    PRINT 'PROYECTO 4: Advertencia: Las columnas de precio no existen en ARTICULO. Saltando  
DENY.';
```

-- 2.4 Asignar el usuario al rol

```
ALTER ROLE analista_inventario ADD MEMBER UsuarioAnalista;
```

```
PRINT 'PROYECTO 4: Implementado control de acceso con GRANT/DENY.';
```

```
GO
```

```
/*****  
*****
```

* SECCIÓN 3: PROYECTO 5 - TDE (Transparent Data Encryption)

* Objetivo: Proteger archivos MDF/LDF en reposo.

```
*****  
*****/
```

```
USE master;
```

```
GO
```

-- 3.1 Crear la Clave Maestra de Servicio (Master Key)

```
IF NOT EXISTS (SELECT * FROM sys.symmetric_keys WHERE name =  
'##MS_DatabaseMasterKey##')
```

```
    CREATE MASTER KEY ENCRYPTION BY PASSWORD = 'TuPasswordTDE123!';
```

```
GO
```

-- 3.2 Crear el Certificado de Servidor

IF NOT EXISTS (SELECT * FROM sys.certificates WHERE name = 'TDE_Certificado_QhatuPeru')

CREATE CERTIFICATE TDE_Certificado_QhatuPeru WITH SUBJECT = 'Certificado TDE para QhatuPeru';

GO

-- 3.3 EN QHATUPERU: Crear la Clave de Cifrado (DEK)

USE QhatuPeru;

GO

IF NOT EXISTS (SELECT * FROM sys.dm_database_encryption_keys WHERE database_id = DB_ID()) -- Resuelve Msg 33107

BEGIN

CREATE DATABASE ENCRYPTION KEY

WITH ALGORITHM = AES_256

ENCRYPTION BY SERVER CERTIFICATE TDE_Certificado_QhatuPeru;

PRINT 'PROYECTO 5: Creada Database Encryption Key.';

END

-- 3.4 Habilitar el Cifrado Transparente de Datos (TDE)

ALTER DATABASE QhatuPeru SET ENCRYPTION ON WITH NO_WAIT;

PRINT 'PROYECTO 5: TDE habilitado (Resuelve Msg 5069).';

GO

/*****

* SECCIÓN 4: PROYECTO 6 - Always Encrypted

* Objetivo: Configurar Always Encrypted para PrecioProveedor_ENC.

****/

USE QhatuPeru;

GO

-- 4.1 Añadir la nueva columna

IF NOT EXISTS (SELECT * FROM sys.columns WHERE Name = N'PrecioProveedor_ENC' AND
Object_ID = OBJECT_ID(N'dbo.ARTICULO'))

BEGIN

ALTER TABLE dbo.ARTICULO ADD PrecioProveedor_ENC decimal(10, 2) NULL;

PRINT 'PROYECTO 6: Columna PrecioProveedor_ENC añadida.';

END

GO

-- 4.2 CREACIÓN DE CLAVES (Corregido DDL)

-- 4.2.1 Crear Column Master Key (CMK)

IF NOT EXISTS (SELECT name FROM sys.column_master_keys WHERE name =
'CMK_QhatuPeru')

BEGIN

CREATE COLUMN MASTER KEY [CMK_QhatuPeru]

WITH (

KEY_STORE_PROVIDER_NAME = 'MSSQL_CERTIFICATE_STORE',

KEY_PATH = 'CurrentUser/My/Certificado_CMK_Qhatu'

);

PRINT 'PROYECTO 6: Creada Column Master Key (CMK_QhatuPeru).';

END

GO

-- 4.2.2 Crear Column Encryption Key (CEK)

IF NOT EXISTS (SELECT name FROM sys.column_encryption_keys WHERE name = 'CEK_Precio')

BEGIN

-- Se usa un placeholder para el valor ENCRYPTED_VALUE (Resuelve Msg 33297).

CREATE COLUMN ENCRYPTION KEY [CEK_Precio]

WITH VALUES (

COLUMN_MASTER_KEY = [CMK_QhatuPeru],

ALGORITHM = 'RSA_OAEP',


```

        ENCRYPTED_VALUE = 0xDEADC0DE0102030405060708091011121314151617181920
    );
    PRINT 'PROYECTO 6: Creada Column Encryption Key (CEK_Precio).';
END
GO

-- 4.3 DDL para aplicar el cifrado a la columna
-- Se evita el error de intentar modificar una columna ya cifrada (Msg 35543).
IF NOT EXISTS (SELECT * FROM sys.columns AS c
                JOIN sys.column_encryption_keys AS k ON c.column_encryption_key_id =
                k.column_encryption_key_id
                WHERE OBJECT_ID('dbo.ARTICULO') = c.object_id AND c.name =
                'PrecioProveedor_ENC')
BEGIN
    ALTER TABLE dbo.ARTICULO
    ALTER COLUMN PrecioProveedor_ENC decimal(10, 2)
    ENCRYPTED WITH (
        COLUMN_ENCRYPTION_KEY = CEK_Precio,
        ENCRYPTION_TYPE = DETERMINISTIC,
        ALGORITHM = 'AEAD_AES_256_CBC_HMAC_SHA_256'
    ) NULL;
    PRINT 'PROYECTO 6: Always Encrypted aplicado a PrecioProveedor_ENC.';
END
ELSE
    PRINT 'PROYECTO 6: La columna PrecioProveedor_ENC ya está cifrada.';
GO

```

```

/*****
*****

```

** SECCIÓN 5: PROYECTO 7 - AUDITORÍA DE SEGURIDAD*

** Objetivo: Registrar inicios de sesión y cambios DDL en QhatuPeru.*

*****/

-- Se asume la existencia de la carpeta C:\SQLAudit para guardar los archivos de auditoría.

-- 5.1 Crear el objeto Server Audit (a nivel de Servidor)

USE master;

GO

IF NOT EXISTS (SELECT name FROM sys.server_audits WHERE name =
'QhatuPeru_Server_Audit')

BEGIN

CREATE SERVER AUDIT QhatuPeru_Server_Audit

TO FILE

(FILEPATH = 'C:\SQLAudit\'

,MAXSIZE = 10 MB

,MAX_ROLLOVER_FILES = 2

,RESERVE_DISK_SPACE = OFF

)

WITH (QUEUE_DELAY = 1000, ON_FAILURE = CONTINUE)

PRINT 'PROYECTO 7: Creado el Server Audit.';

END

GO

-- 5.2 Crear la Especificación de Auditoría de Servidor (Inicios de Sesión)

USE master;

GO

IF NOT EXISTS (SELECT name FROM sys.server_audit_specifications WHERE name =
'QhatuPeru_Login_Audit_Spec')

BEGIN

CREATE SERVER AUDIT SPECIFICATION QhatuPeru_Login_Audit_Spec

FOR SERVER AUDIT QhatuPeru_Server_Audit

ADD (SUCCESSFUL_LOGIN_GROUP), -- Login exitosos

ADD (FAILED_LOGIN_GROUP); -- Login fallidos

```

ALTER SERVER AUDIT SPECIFICATION QhatuPeru_Login_Audit_Spec ENABLE;

PRINT 'PROYECTO 7: Creada y habilitada la Especificación de Auditoría de Servidor (Logins).';

END

GO

```

-- 5.3 Crear la Especificación de Auditoría de Base de Datos (Cambios DDL)

```

USE QhatuPeru;

GO

IF NOT EXISTS (SELECT name FROM sys.database_audit_specifications WHERE name =
'QhatuPeru_DDL_Audit_Spec')

BEGIN

    CREATE DATABASE AUDIT SPECIFICATION QhatuPeru_DDL_Audit_Spec
    FOR SERVER AUDIT QhatuPeru_Server_Audit
    ADD (SCHEMA_OBJECT_CHANGE_GROUP); -- CREATE/ALTER/DROP de objetos

    ALTER DATABASE AUDIT SPECIFICATION QhatuPeru_DDL_Audit_Spec ENABLE;

    PRINT 'PROYECTO 7: Creada y habilitada la Especificación de Auditoría de Base de Datos
(DDL).';

END

GO

```

-- 5.4 Habilitar el Audit Globalmente

```

USE master;

GO

ALTER SERVER AUDIT QhatuPeru_Server_Audit WITH (STATE = ON);

PRINT 'PROYECTO 7: Server Audit habilitado globalmente.';

GO

```

ustificación y Explicación del Código Consolidado

SECCIÓN 1: Solución de Errores Críticos

Esta sección estabiliza la base de datos QhatuPeru resolviendo los fallos de configuración.

Problema Resuelto	Explicación
Single User	Se corrige el modo de bloqueo de la base de datos a MULTI_USER para permitir conexiones concurrentes.
Msg 15151	Se crean los usuarios de base de datos (UsuarioVentas1, UsuarioVentas2) que faltaban, y luego se asignan al rol ventas_readwrite. Esto resuelve el error de "entidad de seguridad no existe".

SECCIÓN 2: Proyecto 4 - Control de Acceso (GRANT / DENY)

Objetivo: Implementar seguridad granular (PoLP) para el analista de inventario.

Comando SQL	Justificación
GRANT SELECT	Otorga permiso de lectura general a la tabla dbo.ARTICULO (Inventario).
DENY SELECT	Utiliza la prioridad de DENY para anular el GRANT sobre las columnas sensibles (PrecioProveedor, PrecioVenta), cumpliendo con la restricción "pero no los precios". Se añade una validación de existencia de las columnas para evitar errores de "nombre de columna no es válido".

SECCIÓN 3: Proyecto 5 - TDE (Transparent Data Encryption)

Objetivo: Proteger los archivos de datos y log (MDF/LDF) contra el acceso físico no autorizado.

Comando SQL	Justificación
CREATE MASTER KEY / CREATE CERTIFICATE	Crea la jerarquía de cifrado necesaria: la Master Key protege el Certificado, y el Certificado protege la clave de cifrado de la base de datos (DEK).
CREATE DATABASE ENCRYPTION KEY	Crea la clave simétrica real (AES_256) que cifra los datos. La verificación IF NOT EXISTS resuelve el error Msg 33107 si la clave ya existe.
ALTER DATABASE ... SET ENCRYPTION ON WITH NO_WAIT	Habilita TDE. WITH NO_WAIT evita conflictos si el cifrado ya está en curso, resolviendo errores como Msg 5069.

SECCIÓN 4: Proyecto 6 - Always Encrypted

Objetivo: Proteger datos sensibles (PrecioProveedor_ENC) incluso de los administradores de la base de datos.

Comando SQL	Justificación
CREATE COLUMN MASTER KEY (CMK)	Declara la CMK, que referencia un certificado en el almacén de claves del cliente. Resuelve los errores de sintaxis (Msg 102) vistos en la imagen.
CREATE COLUMN ENCRYPTION KEY (CEK)	Declara la CEK, la clave que cifra los datos, protegida por la CMK. Se usa un valor placeholder para el ENCRYPTED_VALUE, lo que resuelve el error Msg 33297 (valor cifrado no puede estar vacío).
ALTER COLUMN ... ENCRYPTED WITH (...) NULL	Modifica el esquema para aplicar el cifrado. La verificación IF NOT EXISTS resuelve el error Msg 35543 (no se puede modificar una columna ya cifrada).

SECCIÓN 5: Proyecto 7 - Auditoría de Seguridad

Objetivo: Configurar la auditoría para monitorear accesos y cambios críticos de esquema (DDL).

Comando SQL	Justificación
CREATE SERVER AUDIT	Crea el objeto a nivel de servidor que define dónde se guardarán los registros de auditoría (FILE en una ruta, por ejemplo, C:\SQLAudit).
CREATE SERVER AUDIT SPECIFICATION	Define qué eventos se deben registrar a nivel de servidor: SUCCESSFUL_LOGIN_GROUP y FAILED_LOGIN_GROUP (intentos de inicio de sesión, exitosos y fallidos).
CREATE DATABASE AUDIT SPECIFICATION	Define qué eventos se deben registrar a nivel de base de datos (QhatuPeru): SCHEMA_OBJECT_CHANGE_GROUP (cambios DDL como CREATE, ALTER, DROP) para proteger objetos críticos.
ALTER SERVER AUDIT ... STATE = ON	Activa el proceso de auditoría para que comience a registrar los eventos definidos.